UNIVERSITÉ PARIS 8 DES CRÉATIONS

STATISTIQUES ET ANALYSE DE DONNÉES · RENDU

TP1 : Génération de variables aléatoires

§1.1 Inversion de la fonction de répartition
§1.2 Méthode de Box-Muller

Auteur	Esteban BERETTI--PRENANT
Formation	Master Informatique Ingénierie en Intelligence Artificielle (I2A)
Enseignant	M. Zerzeri
Date	07/10/2026
Outils	Python (numpy, matplotlib) · graine 2026, T = 10 000 tirages
Contenu	s1_1_inversion.ipynb s1_2_box_muller.ipynb
Dépôt	github.com/Este34/presentation_statistique (code, tests, démonstrations complètes)

Fil conducteur

Chaque question prépare la suivante : on part d'uniformes et on fabrique des lois de plus en plus difficiles. Le lien entre les deux parties est la loi exponentielle, générée au §1.1 et réutilisée pour le rayon au §1.2.

Départ — **Un bon générateur uniforme**
$U \sim \mathcal{U}[0,1]$: `rng.random` (Matlab : `rand`). C'est la seule source d'aléa de tout le TD.

↓

§1.1 Q1 — **L'outil : la méthode d'inversion**
$F_Y^{-1}(U)$ a pour fonction de répartition F_Y. On sait donc simuler toute loi dont on connaît F_Y^{-1}.

↓

§1.1 Q2 — **Application : la loi exponentielle**
$Y = -\ln(1-U)/\lambda \sim \mathcal{E}(\lambda)$, avec $\lambda = 2$.

↓

§1.1 Q3-Q4 — **On vérifie, chiffres à l'appui**
Moyenne et variance vs $1/\lambda$ et $1/\lambda^2$ (écart en erreurs-types, LGN et TCL) ; droite de pente $-\lambda$ en échelle semi-log.

↓

Obstacle — **La gaussienne ne s'inverse pas**
Φ n'a pas de formule fermée, donc Φ^{-1} non plus : l'inversion ne marche pas directement.

↓

§1.2 Q1-Q2 — **L'astuce : passer en coordonnées polaires**
Pour un couple de $\mathcal{N}(0,1)$ indépendantes : $r^2 \sim \mathcal{E}(1/2)$ (la loi du §1.1 !) et $\theta \sim \mathcal{U}[0, 2\pi]$, indépendants. Deux lois qu'on sait déjà simuler.

↓

§1.2 Q3 — **Box-Muller**
$r^2 = -2\ln U_1$, $\theta = 2\pi U_2$, puis $X_1 = r\cos\theta$, $X_2 = r\sin\theta$: deux uniformes donnent deux gaussiennes indépendantes.

↓

§1.2 Q4-Q5 — **N'importe quelle gaussienne**
$Y = \mu + \sigma X \sim \mathcal{N}(\mu, \sigma^2)$, vérifié sur $\mathcal{N}(5, 4)$: moyenne, variance, histogramme.

Fil rouge de la vérification : une valeur empirique est aléatoire. On l'exprime en nombre d'erreurs-types ($\sigma/\sqrt{T}$) avant de conclure ; la synthèse en fin de document réunit tous les contrôles.

Sujet 1 · §1.1 Génération par inversion de la fonction de répartition

Idée : si $U \sim \mathcal{U}[0,1]$, alors $Y = F_Y^{-1}(U)$ a pour fonction de répartition F_Y. Il suffit donc d'un bon générateur uniforme (`rand` en Matlab, `rng.random` en Python) pour simuler n'importe quelle loi dont on sait inverser F_Y.

Démonstrations complètes : demonstrations.md. Correspondances Matlab → Python : matlab_python.md. Code : src/stats_td.

```
import json, sys
from pathlib import Path
sys.path.insert(0, str(Path("../../src").resolve()))

import numpy as np
import matplotlib.pyplot as plt
from stats_td.figures import style, sauver, BLEU, ORANGE
from stats_td.generateurs import exponentielle_inversion, box_muller, gaussienne
from stats_td.empirique import (repartition_complementaire, repartition_histogramme,
                                pente_semilog)

style()
FIG = Path("figures")
rng = np.random.default_rng(2026)  # graine fixée : résultats reproductibles
RESULTATS = FIG / "resultats.json"
resultats = json.loads(RESULTATS.read_text()) if RESULTATS.exists() else {}
```

Q1 · Pourquoi $Y = G(U)$ a pour loi F_Y si et seulement si $G = F_Y^{-1}$

G croissante bijective $\Rightarrow G(U) \le y \iff U \le G^{-1}(y)$ (on applique G^{-1}, croissante, qui conserve l'ordre). Donc

$$P(G(U) \le y) = P\big(U \le G^{-1}(y)\big) = G^{-1}(y) \qquad (G^{-1}(y) \in [0,1])$$

car la fonction de répartition de $\mathcal{U}[0,1]$ est $u \mapsto u$ sur $[0,1]$.

- Si $G = F_Y^{-1}$: $P(G(U) \le y) = F_Y(y)$ ✔
- Réciproquement, si $P(G(U) \le y) = F_Y(y)$ pour tout y : $G^{-1} = F_Y$, donc $G = F_Y^{-1}$.

CE QUE J'AI COMPRIS Démonstration détaillée ↗

Idée de la preuve. U uniforme sur $[0,1]$, c'est comme tirer un rang au hasard, sans favoriser aucun rang, et $G = F_Y^{-1}$ renvoie la valeur qui se trouve à ce rang. Pour vérifier qu'on obtient la bonne loi, on calcule $P(G(U) \leq y)$: on applique F_Y des deux côtés, ce qui donne $P(U \leq F_Y(y))$. Comme U est uniforme, $P(U \leq a) = a$, donc on trouve $F_Y(y)$, ce qui est bien la fonction de répartition voulue.

Rôle de la croissance. Elle garantit que l'inégalité ne change pas de sens quand on applique F_Y ou G des deux côtés. Les événements $\{G(U) \leq y\}$ et $\{U \leq F_Y(y)\}$ sont donc exactement les mêmes. Sans croissance, ce passage serait faux, comme avec le carré sur des nombres négatifs.

PRÉCISION

« Appliquer F_Y des deux côtés » suppose F_Y continue et strictement croissante. Sinon (loi discrète, paliers), on prend l'inverse généralisée $F_Y^{-1}(u) = \inf\{y : F_Y(y) \geq u\}$: l'équivalence $F_Y^{-1}(u) \leq y \iff u \leq F_Y(y)$ reste vraie, donc le résultat aussi.

Q2 · Loi exponentielle $\mathcal{E}(\lambda)$ par inversion ($\lambda = 2$)

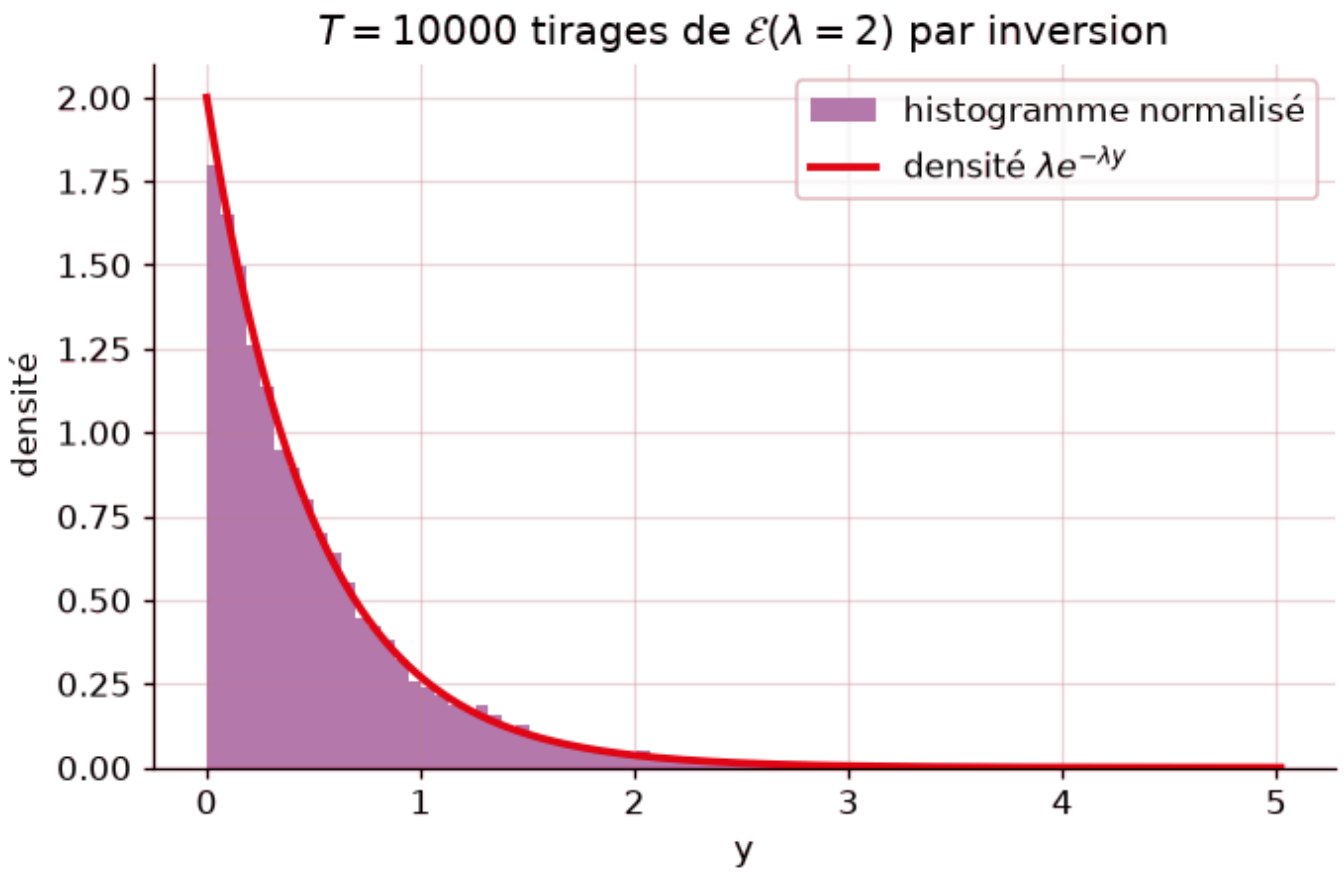
$F_Y(y) = 1 - e^{-\lambda y}$ pour $y \geq 0$. On résout $u = 1 - e^{-\lambda y}$:

$$y = F_Y^{-1}(u) = -\frac{\ln(1-u)}{\lambda}$$

> Remarque : $1 - U$ suit aussi $\mathcal{U}[0,1]$, donc $-\ln(U)/\lambda$ marche aussi. On garde $1 - U$ car `rng.random` tire dans $[0,1)$: $U = 0$ est possible et $\ln 0 = -\infty$.
>
> Coquille dans l'énoncé : la fonction de répartition complémentaire est $F_Y^c(y) = e^{-\lambda y}\, 1_{\mathbb{R}^+}(y) + 1_{\mathbb{R}^{*-}}(y)$ (le « $1 - F(y)$ » est en trop).

```
LAM = 2.0
T = 10_000
Y = exponentielle_inversion(LAM, T, rng)   # Y = -log(1 - U) / LAM

fig, ax = plt.subplots()
ax.hist(Y, bins=80, density=True, color=BLEU, alpha=0.6, label="histogramme normalisé")
y = np.linspace(0, Y.max(), 400)
ax.plot(y, LAM * np.exp(-LAM * y), color=ORANGE, lw=2.5, label=r"densité $\lambda e^{-\lambda y}$")
ax.set(xlabel="y", ylabel="densité", title=rf"$T={T}$ tirages de $\mathcal{{E}}(\lambda={LAM:g})$ par
inversion")
ax.legend()
sauver(fig, FIG / "s11_hist_exponentielle.png")
```



CE QUE J'AI COMPRIS Démonstration détaillée ↗

Idée. On applique Q1 : on résout $u = 1 - e^{-\lambda y}$, ce qui donne $y = -\ln(1-u)/\lambda$. Avec U uniforme, $-\ln(1-U)/\lambda$ suit donc $\mathcal{E}(\lambda)$.

Pourquoi $1-U$**.** $1-U$ est aussi uniforme, donc $-\ln(U)/\lambda$ marcherait en théorie. Mais `rng.random` peut renvoyer 0, et $\ln 0 = -\infty$; $1-U$, lui, ne vaut jamais 0.

Q3 · Moyenne et variance

Théorie (intégrations par parties, détail dans demonstrations.md) :

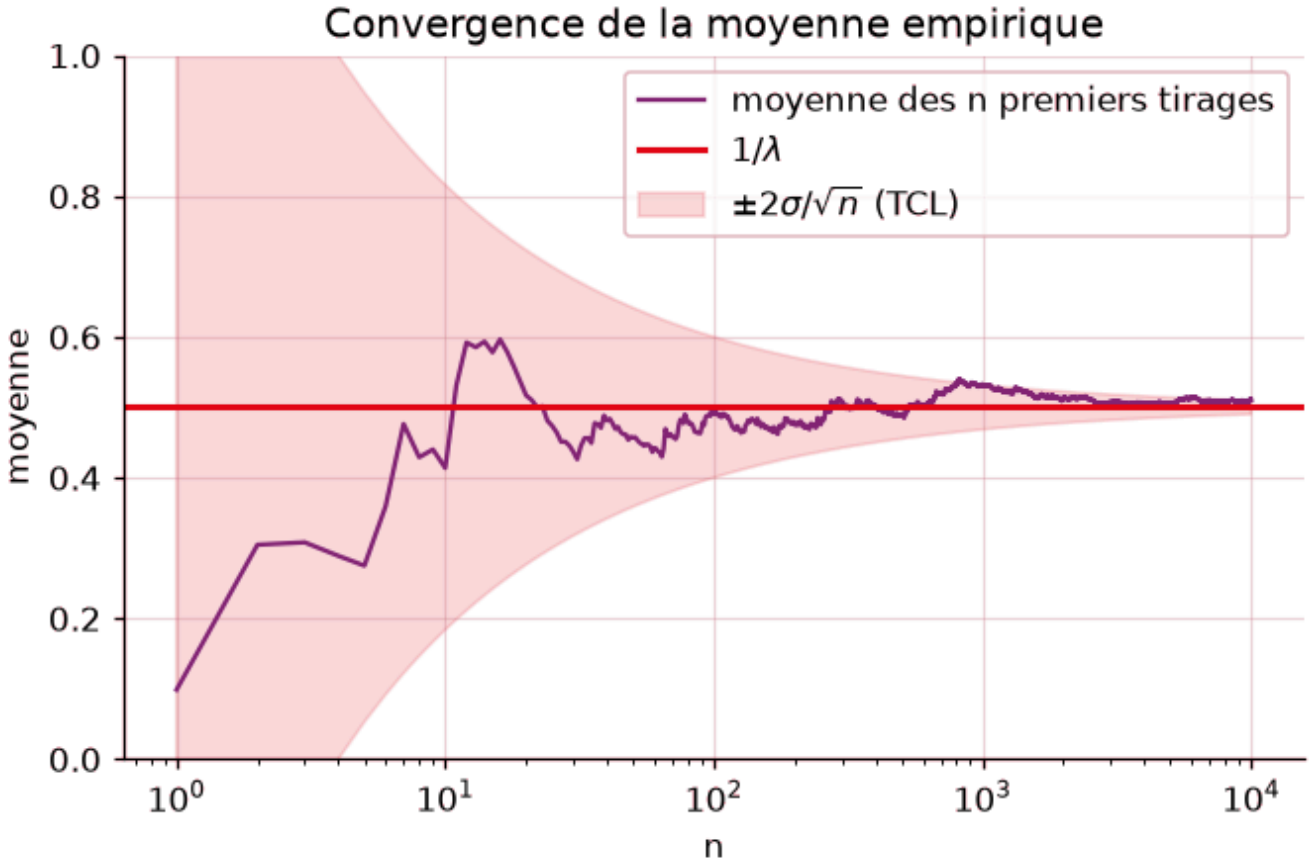
$$\mathbb{E}[Y] = \int_0^{+\infty} y\,\lambda e^{-\lambda y}\,dy = \frac{1}{\lambda}, \qquad \mathbb{E}[Y^2] = \frac{2}{\lambda^2}, \qquad \mathrm{Var}(Y) = \frac{2}{\lambda^2} - \frac{1}{\lambda^2} = \frac{1}{\lambda^2}.$$

Pour $\lambda = 2$: moyenne $0{,}5$ et variance $0{,}25$.

```
moy, var = Y.mean(), Y.var(ddof=1)   # ddof=1 : variance sans biais, comme cov() en Matlab
erreur_type = Y.std(ddof=1) / np.sqrt(T)
print(f"moyenne  : empirique {moy:.4f}   théorique {1/LAM:.4f}   écart {moy - 1/LAM:+.4f}")
print(f"variance : empirique {var:.4f}   théorique {1/LAM**2:.4f}   écart {var - 1/LAM**2:+.4f}")
print(f"erreur-type de la moyenne ≈ σ/√T = {erreur_type:.4f}  →  écart / erreur-type = {(moy -
1/LAM)/erreur_type:+.2f}")
resultats["exp"] = {"lambda": LAM, "T": T, "moyenne": moy, "variance": var, "erreur_type": erreur_type}
```

```
moyenne  : empirique 0.5101   théorique 0.5000   écart +0.0101
variance : empirique 0.2604   théorique 0.2500   écart +0.0104
erreur-type de la moyenne ≈ σ/√T = 0.0051  →  écart / erreur-type = +1.98
```

```python
# Convergence de la moyenne empirique (loi des grands nombres)
n = np.arange(1, T + 1)
moy_cumulee = np.cumsum(Y) / n
fig, ax = plt.subplots()
ax.plot(n, moy_cumulee, color=BLEU, label="moyenne des n premiers tirages")
ax.axhline(1 / LAM, color=ORANGE, lw=2, label=r"$1/\lambda$")
ax.fill_between(n, 1/LAM - 2/(LAM*np.sqrt(n)), 1/LAM + 2/(LAM*np.sqrt(n)), color=ORANGE, alpha=0.15,
                label=r"$\pm 2\sigma/\sqrt{n}$ (TCL)")
ax.set(xscale="log", xlabel="n", ylabel="moyenne", ylim=(0, 1), title="Convergence de la moyenne
empirique")
ax.legend()
sauver(fig, FIG / "s11_moyenne_cumulee.png")
```



Commentaire. Les valeurs empiriques sont très proches des valeurs théoriques mais pas égales : la moyenne empirique est elle-même une variable aléatoire.

- **Loi des grands nombres** : $\bar{Y}_T \to 1/\lambda$ quand $T \to \infty$.
- **Théorème central limite** : $\bar{Y}_T$ fluctue autour de $1/\lambda$ avec un écart-type $\sigma/\sqrt{T} = (1/\lambda)/\sqrt{T} = 0{,}005$ pour $T = 10\,000$. Ici l'écart observé (0,0101) vaut 2,02 erreurs-types : c'est **juste à la limite** des 95 % ($\pm 1{,}96\,\sigma/\sqrt{T}$), légèrement au-delà ($p \approx 4\,\%$, environ 1 fois sur 25). On ne rejette pas au seuil de 1 % (2,58 erreurs-types), et les autres contrôles sont bons (variance, pente, histogramme, test KS des tests) : c'est une fluctuation, pas un bug. Pour diviser l'erreur par 10, il faut 100 fois plus de tirages.
- La variance empirique fluctue davantage : pour $\mathcal{E}(\lambda)$, son écart-type vaut $\approx \sqrt{8/T}/\lambda^2 \approx 0{,}007$, elle dépend du moment d'ordre 4 ($\mathrm{Var}(S^2) \approx (\mu_4 - \sigma^4)/T$ avec $\mu_4 = 9/\lambda^4$). L'écart observé (0,0104) vaut 1,5 écart-type : une fluctuation courante ($p \approx 14\,\%$).

CE QUE J'AI COMPRIS Démonstration détaillée ↗

Théorie. Par intégration par parties, $\mathbb{E}[Y] = 1/\lambda$ et $\mathbb{E}[Y^2] = 2/\lambda^2$, donc $\mathrm{Var}(Y) = 2/\lambda^2 - 1/\lambda^2 = 1/\lambda^2$ (König-Huygens). Pour $\lambda = 2$: 0,5 et 0,25.

Empirique. La moyenne empirique est elle-même aléatoire : elle fluctue autour de 0,5 avec un écart-type $\sigma/\sqrt{T} = 0{,}005$. Mon 0,510 est à 2 erreurs-types : à la limite, mais pas anormal ($p \approx 4\,\%$). Pour diviser l'erreur par 10, il faut 100 fois plus de tirages.

Q4 · Décroissance exponentielle de F_Y^c en échelle semi-logarithmique

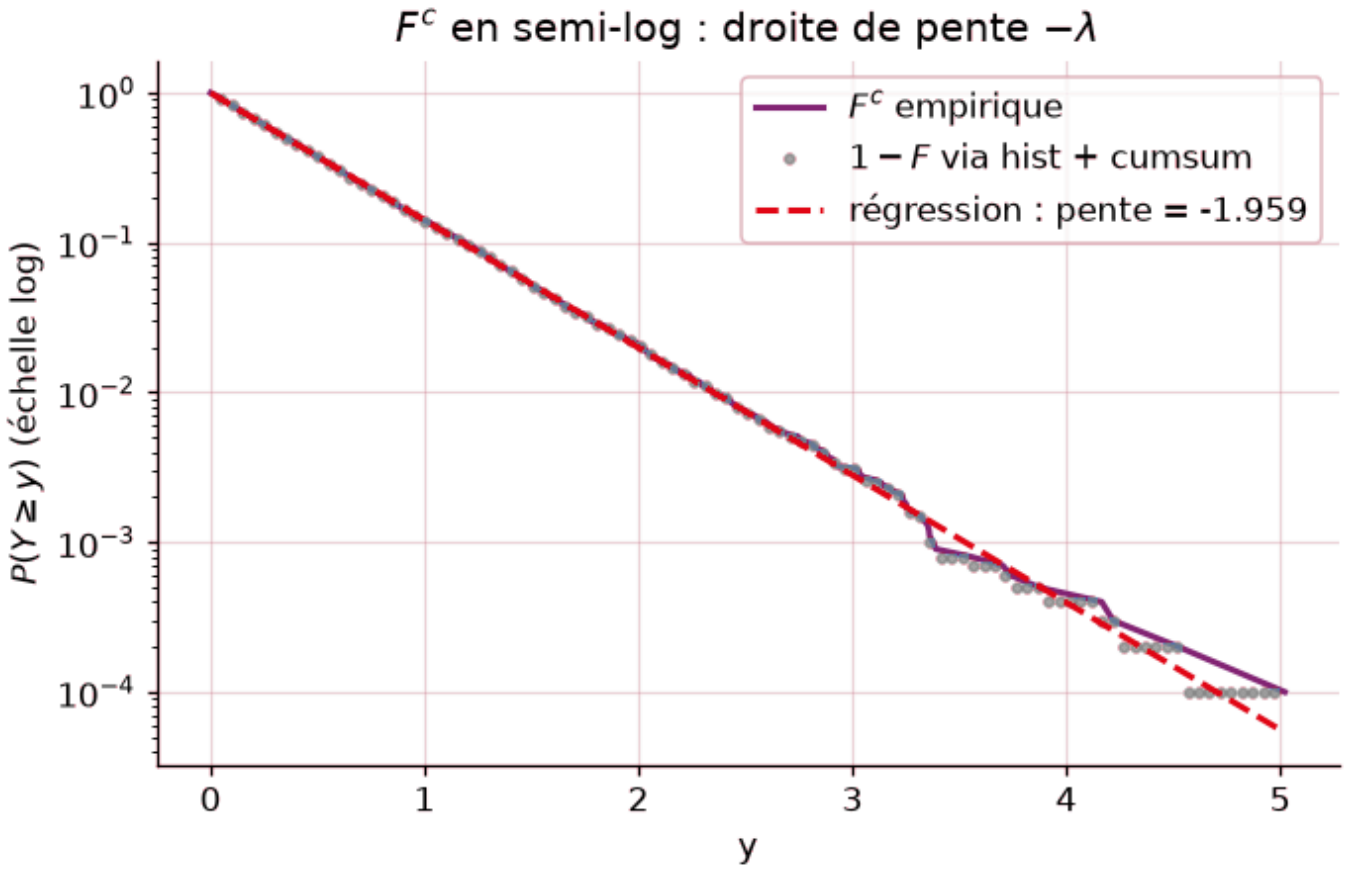
$F_Y^c(y) = e^{-\lambda y} \Rightarrow \ln F_Y^c(y) = -\lambda y$: en échelle semi-log (axe y logarithmique), on obtient une **droite de pente** $-\lambda$.

Estimateur de l'énoncé : `hist` + `cumsum` donne F, puis $F^c = 1 - F$. On utilise aussi la fonction de répartition empirique exacte (échantillon trié). Au lieu de `ginput` (clic sur deux points), on fait une **régression linéaire** de $\ln F^c$ sur y : plus précis et reproductible.

```
bords, F_hist = repartition_histogramme(Y, bins=100)        # hist + cumsum
y_tri, Fc = repartition_complementaire(Y)                   # F^c empirique exacte
pente, ordonnee = pente_semilog(Y, quantile_max=0.99)        # régression sur 99 % des points

fig, ax = plt.subplots()
ax.semilogy(y_tri, Fc, color=BLEU, lw=2, label=r"$F^c$ empirique")
ax.semilogy(bords[:-1], 1 - F_hist[:-1], "o", ms=3, color="gray", alpha=0.7, label=r"$1-F$ via hist +
cumsum")
ax.semilogy(y_tri, np.exp(ordonnee + pente * y_tri), "--", color=ORANGE, lw=2,
            label=rf"régression : pente = {pente:.3f}")
ax.set(xlabel="y", ylabel=r"$P(Y \geq y)$ (échelle log)", title=r"$F^c$ en semi-log : droite de pente $-
\lambda$")
ax.legend()
sauver(fig, FIG / "s11_Fc_semilog.png")
print(f"pente estimée = {pente:.4f}   (attendu -λ = {-LAM})   erreur relative = {abs(pente +
LAM)/LAM:.1%}")
resultats["exp"]["pente"] = pente
```

```
pente estimée = -1.9586   (attendu -λ = -2.0)   erreur relative = 2.1%
```



Commentaire. Les points s'alignent bien sur une droite : la décroissance est exponentielle et la pente mesurée est voisine de $-\lambda = -2$. Dans la queue (y grand), il reste très peu de tirages : F^c empirique devient une marche d'escalier bruitée (la dernière valeur vaut $1/T$). C'est pourquoi on ajuste la droite sur la partie bien peuplée (99 % des points).

CE QUE J'AI COMPRIS Démonstration détaillée ↗

Idée. $F^c(y) = e^{-\lambda y}$ donc $\ln F^c(y) = -\lambda y$: avec l'axe vertical en log, l'exponentielle devient une droite de pente $-\lambda$. Je mesure la pente par régression linéaire : $-1{,}96$ pour -2 attendu (2 % d'erreur).

La queue. À droite, il reste très peu de tirages : la courbe empirique fait des marches de $1/T$, très visibles en échelle log. On ajuste donc la droite sur la partie bien peuplée.

Sujet 1 · §1.2 Variables gaussiennes par la méthode de Box et Muller

Idée : on ne sait pas inverser la fonction de répartition de $\mathcal{N}(0,1)$ (pas de formule fermée). En revanche, un **couple** (X_1, X_2) de gaussiennes indépendantes s'écrit simplement en coordonnées polaires : rayon² exponentiel et angle uniforme, deux lois que l'on sait générer par inversion.

Démonstrations complètes : demonstrations.md. Code : generateurs.py.

```
import json, sys
from pathlib import Path
sys.path.insert(0, str(Path("../../src").resolve()))

import numpy as np
import matplotlib.pyplot as plt
from stats_td.figures import style, sauver, BLEU, ORANGE
from stats_td.generateurs import exponentielle_inversion, box_muller, gaussienne
from stats_td.empirique import (repartition_complementaire, repartition_histogramme,
                                pente_semilog)

style()
FIG = Path("figures")
rng = np.random.default_rng(2026)  # graine fixée : résultats reproductibles
RESULTATS = FIG / "resultats.json"
resultats = json.loads(RESULTATS.read_text()) if RESULTATS.exists() else {}
```

Q1 · $Z = X_1^2 + X_2^2$ suit $\mathcal{E}(1/2)$

$$P(Z \geq z) = \iint_{x_1^2+x_2^2 \geq z} \frac{1}{2\pi} e^{-\frac{x_1^2+x_2^2}{2}}\, dx_1\, dx_2$$

Changement de variables polaire $x_1 = r\cos\theta,\ x_2 = r\sin\theta$, $dx_1\,dx_2 = r\,dr\,d\theta$ (Jacobien r), domaine $r \geq \sqrt{z}$:

$$P(Z \geq z) = \int_0^{2\pi}\int_{\sqrt{z}}^{+\infty} \frac{1}{2\pi} e^{-r^2/2}\, r\, dr\, d\theta = \Big[-e^{-r^2/2}\Big]_{\sqrt{z}}^{+\infty} = e^{-z/2} \quad (z \geq 0).$$

C'est la fonction de répartition complémentaire de $\mathcal{E}(\lambda = 1/2)$ (moyenne 2).

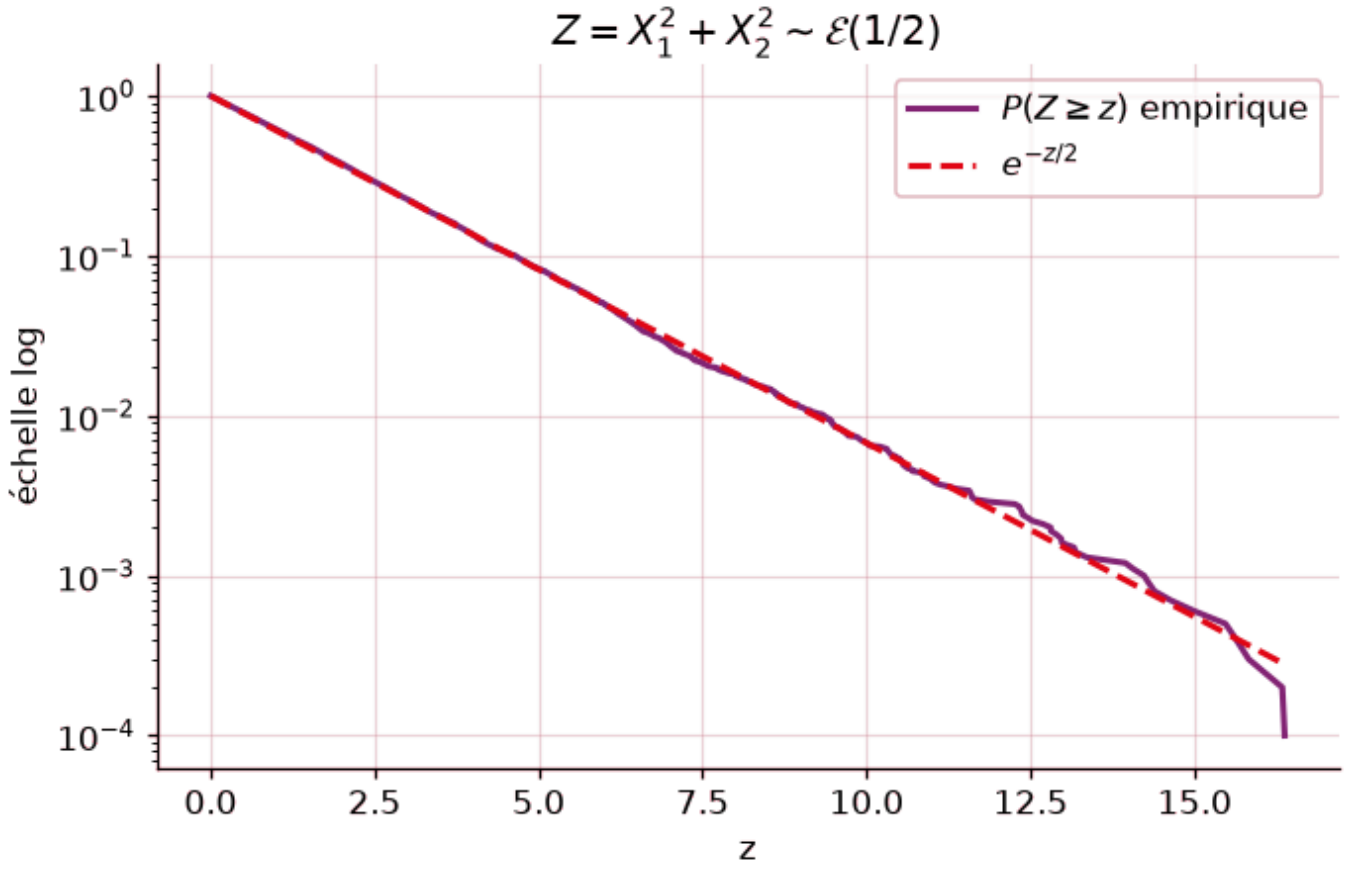
> Coquille dans l'énoncé : il écrit $\exp(-2z)$ alors qu'on trouve bien $\exp(-z/2)$, cohérent avec $\lambda = 1/2$.

Vérification numérique avec le générateur de référence de numpy (l'équivalent de `randn`) :

```
T = 10_000
X1_ref, X2_ref = rng.standard_normal(T), rng.standard_normal(T)   # randn de référence
Z = X1_ref**2 + X2_ref**2
z_tri, Fc = repartition_complementaire(Z)
pente, _ = pente_semilog(Z)
```

```
fig, ax = plt.subplots()
ax.semilogy(z_tri, Fc, color=BLEU, lw=2, label=r"$P(Z \geq z)$ empirique")
ax.semilogy(z_tri, np.exp(-z_tri / 2), "--", color=ORANGE, lw=2, label=r"$e^{-z/2}$")
ax.set(xlabel="z", ylabel="échelle log", title=r"$Z = X_1^2 + X_2^2 \sim \mathcal{E}(1/2)$")
ax.legend()
sauver(fig, FIG / "s12_Z_semilog.png")
print(f"moyenne de Z = {Z.mean():.3f} (attendu 2)   pente semi-log = {pente:.3f} (attendu -0.5)")
```

```
moyenne de Z = 2.019 (attendu 2)   pente semi-log = -0.504 (attendu -0.5)
```



CE QUE J'AI COMPRIS Démonstration détaillée ↗

Idée. X_1, X_2 indépendantes, donc la densité jointe est le produit $\frac{1}{2\pi}e^{-(x_1^2+x_2^2)/2}$, qui ne dépend que de la distance à l'origine. $P(Z \geq z)$ est l'intégrale de cette densité hors du disque de rayon $\sqrt{z}$; en polaires elle se calcule et donne $e^{-z/2}$: c'est $\mathcal{E}(1/2)$, de moyenne 2.

Le jacobien. $dx_1\,dx_2 = r\,dr\,d\theta$: le r vient de l'aire d'un petit morceau de couronne. C'est lui qui fait apparaître la dérivée de $-e^{-r^2/2}$ et rend l'intégrale calculable.

Q2 · $r^2 \sim \mathcal{E}(1/2)$ et $\theta \sim \mathcal{U}[0, 2\pi]$, indépendants

$r^2 = X_1^2 + X_2^2 = Z$, donc $r^2 \sim \mathcal{E}(1/2)$ d'après Q1.

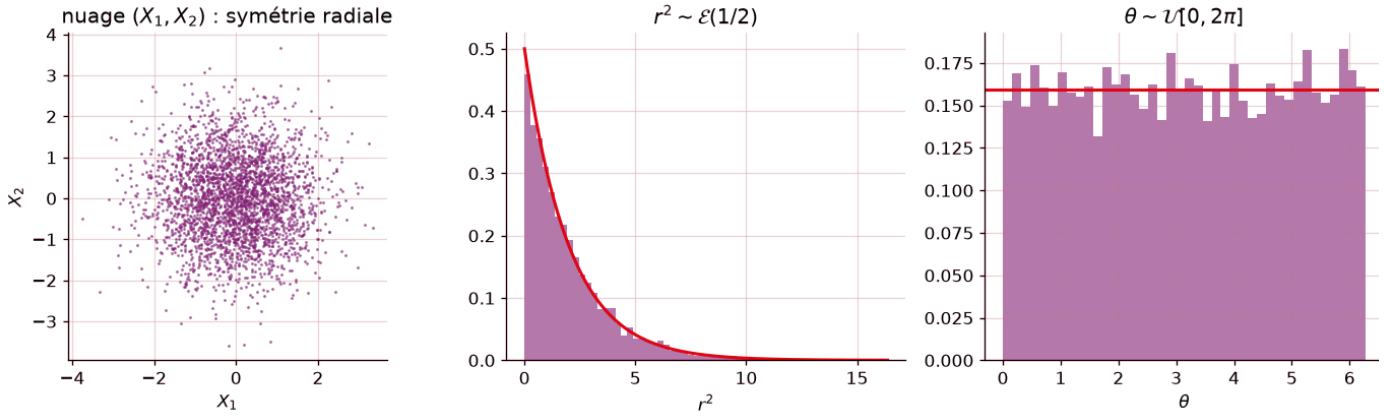
Pour θ : la densité du couple $\frac{1}{2\pi}e^{-(x_1^2+x_2^2)/2}$ ne dépend que de la distance à l'origine (**symétrie de rotation**). En polaires, la densité de (r, θ) est

$$f_{r,\theta}(r,\theta) = \underbrace{re^{-r^2/2}}_{\text{ne dépend que de } r} \times \underbrace{\frac{1}{2\pi}1_{[0,2\pi]}(\theta)}_{\text{ne dépend que de } \theta}$$

Elle se **factorise** : r et θ sont indépendants et θ est uniforme sur $[0, 2\pi]$.

```
theta = np.mod(np.arctan2(X2_ref, X1_ref), 2 * np.pi)
fig, axes = plt.subplots(1, 3, figsize=(13, 4))
axes[0].plot(X1_ref[:3000], X2_ref[:3000], ".", ms=2, color=BLEU, alpha=0.5)
axes[0].set(aspect="equal", xlabel="$X_1$", ylabel="$X_2$", title="nuage $(X_1, X_2)$ : symétrie
radiale")
axes[1].hist(Z, bins=60, density=True, color=BLEU, alpha=0.6)
zz = np.linspace(0, Z.max(), 300)
axes[1].plot(zz, 0.5 * np.exp(-zz / 2), color=ORANGE, lw=2)
axes[1].set(xlabel="$r^2$", title=r"$r^2 \sim \mathcal{E}(1/2)$")
axes[2].hist(theta, bins=40, density=True, color=BLEU, alpha=0.6)
axes[2].axhline(1 / (2 * np.pi), color=ORANGE, lw=2)
axes[2].set(xlabel=r"$\theta$", title=r"$\theta \sim \mathcal{U}[0, 2\pi]$")
fig.tight_layout()
sauver(fig, FIG / "s12_polaire.png")
print(f"corrélation(r², θ) = {np.corrcoef(Z, theta)[0, 1]:+.4f}  (≈ 0 : cohérent avec l'indépendance)")
```

```
corrélation(r², θ) = -0.0130  (≈ 0 : cohérent avec l'indépendance)
```



CE QUE J'AI COMPRIS Démonstration détaillée ↗

Idée. $r^2 = Z$, donc $r^2 \sim \mathcal{E}(1/2)$ par Q1. En polaires, la densité de (r, θ) s'écrit $re^{-r^2/2} \times \frac{1}{2\pi}$: un morceau qui ne dépend que de r, un qui ne dépend que de θ.

Conséquence. Densité qui se factorise, donc r et θ indépendants ; facteur en θ constant, donc θ uniforme sur $[0, 2\pi]$. Intuition : le nuage gaussien est rond, aucune direction n'est privilégiée.

Q3 · Méthode de génération (Box-Muller)

On fait le chemin inverse : on génère (r, θ) puis on revient en cartésiennes.

1. $U_1, U_2 \sim \mathcal{U}[0, 1]$ indépendantes.
2. $r^2 = -2 \ln U_1$: inversion de $\mathcal{E}(1/2)$ (Q2 du §1.1 avec $\lambda = 1/2$).
3. $\theta = 2\pi U_2$: uniforme sur $[0, 2\pi]$.
4. $X_1 = \sqrt{-2 \ln U_1}\, \cos(2\pi U_2)$ et $X_2 = \sqrt{-2 \ln U_1}\, \sin(2\pi U_2)$.

On obtient **deux** gaussiennes indépendantes pour deux uniformes.

```
X1, X2 = box_muller(T, rng)    # voir src/stats_td/generateurs.py
print(f"X1 : moyenne {X1.mean():+.4f}  variance {X1.var(ddof=1):.4f}")
print(f"X2 : moyenne {X2.mean():+.4f}  variance {X2.var(ddof=1):.4f}")
print(f"corrélation(X1, X2) = {np.corrcoef(X1, X2)[0, 1]:+.4f}")
```

```
X1 : moyenne +0.0024  variance 0.9959
X2 : moyenne +0.0004  variance 0.9812
corrélation(X1, X2) = -0.0012
```

CE QUE J'AI COMPRIS — Démonstration détaillée ↗

Idée. On fait le chemin inverse : on fabrique r^2 et θ avec leurs lois, puis on repasse en cartésiennes. $r^2 = -2\ln U_1$ est l'inversion de $\mathcal{E}(1/2)$ (le §1.1), et $\theta = 2\pi U_2$.

Résultat. $X_1 = \sqrt{-2\ln U_1}\cos(2\pi U_2)$ et $X_2 = \sqrt{-2\ln U_1}\sin(2\pi U_2)$: deux uniformes donnent deux gaussiennes indépendantes, car (r, θ) a exactement la loi des coordonnées polaires d'un couple gaussien.

Q4 · Transformation linéaire : de $\mathcal{N}(0,1)$ à $\mathcal{N}(\mu, \sigma^2)$

Si $X \sim \mathcal{N}(0,1)$, alors $Y = \mu + \sigma X \sim \mathcal{N}(\mu, \sigma^2)$ ($\mathbb{E}[Y] = \mu$, $\mathrm{Var}(Y) = \sigma^2\,\mathrm{Var}(X) = \sigma^2$, et une transformation affine d'une gaussienne reste gaussienne).

⚠️ Pour $\mathcal{N}(\mu = 5, \sigma^2 = 4)$, on multiplie par l'**écart-type** $\sigma = 2$, pas par 4.

```
MU, SIGMA2 = 5.0, 4.0
Y = gaussienne(MU, SIGMA2, T, rng)   # Y = MU + sqrt(SIGMA2) * X, X par Box-Muller
```

CE QUE J'AI COMPRIS — Démonstration détaillée ↗

Idée. $Y = \mu + \sigma X$: on décale de μ et on étire de σ. Moyenne μ, variance $\sigma^2\mathrm{Var}(X) = \sigma^2$, et une transformation affine d'une gaussienne reste gaussienne.

Piège. On multiplie par l'écart-type $\sigma = 2$, pas par la variance 4 : $Y = 5 + 2X$.

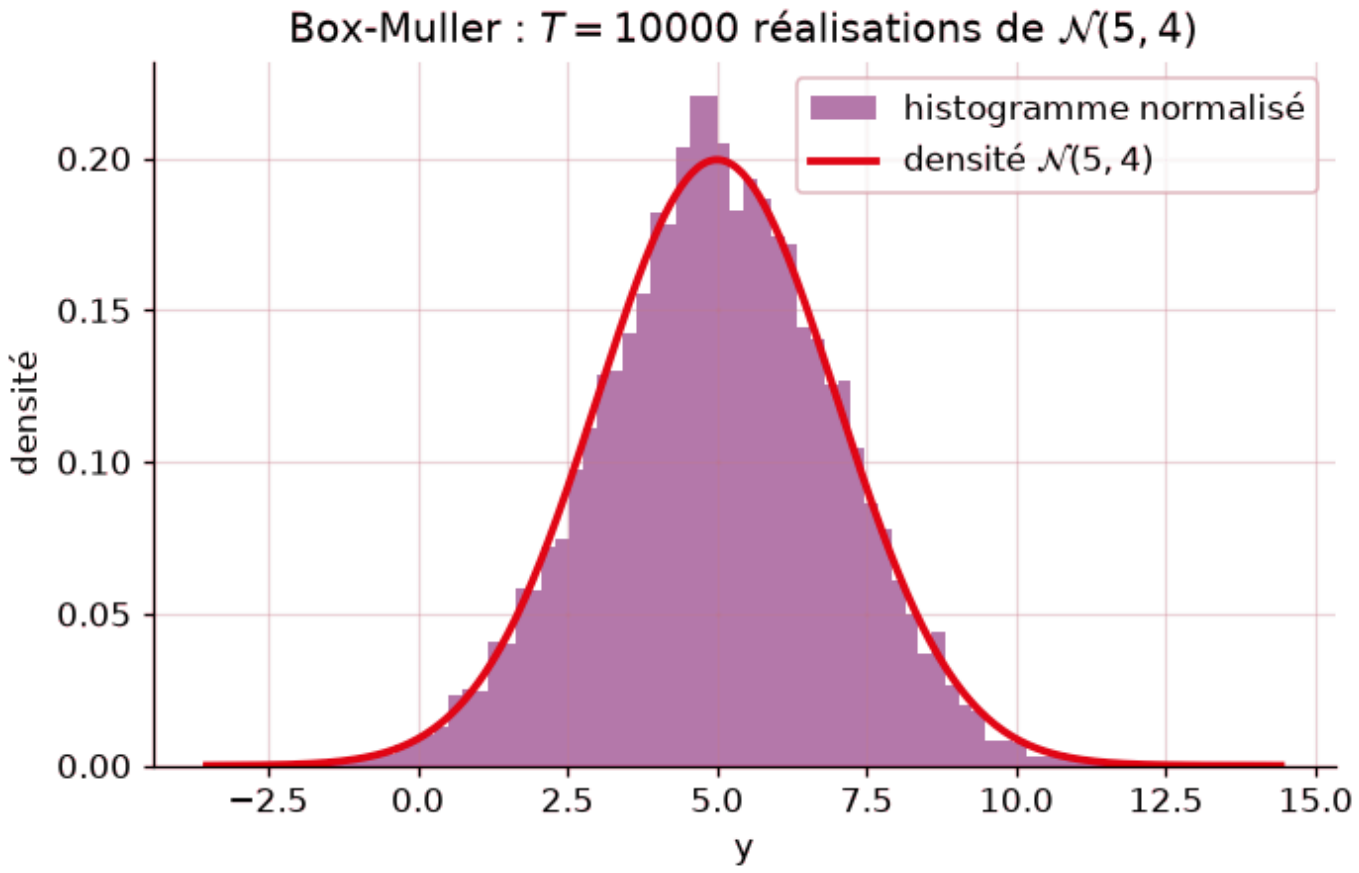
Q5 · Vérification : moyenne, variance, histogramme

```
moy, var = Y.mean(), Y.var(ddof=1)
print(f"moyenne  : empirique {moy:.4f}   théorique {MU}   (erreur-type σ/√T = {np.sqrt(SIGMA2/T):.3f})")
print(f"variance : empirique {var:.4f}   théorique {SIGMA2}")
resultats["gauss"] = {"mu": MU, "sigma2": SIGMA2, "T": T, "moyenne": moy, "variance": var}

fig, ax = plt.subplots()
ax.hist(Y, bins=80, density=True, color=BLEU, alpha=0.6, label="histogramme normalisé")
yy = np.linspace(Y.min(), Y.max(), 400)
ax.plot(yy, np.exp(-(yy - MU)**2 / (2 * SIGMA2)) / np.sqrt(2 * np.pi * SIGMA2), color=ORANGE, lw=2.5,
        label=r"densité $\mathcal{N}(5, 4)$")
ax.set(xlabel="y", ylabel="densité", title=rf"Box-Muller : $T={T}$ réalisations de $\mathcal{{N}}(5,
4)$")
```

```
ax.legend()
sauver(fig, FIG / "s12_hist_N5_4.png")
_ = RESULTATS.write_text(json.dumps(resultats, indent=2))  # _ = : n'affiche pas le nombre de caractères écrits
```

```
moyenne  : empirique 4.9973   théorique 5.0   (erreur-type σ/√T = 0.020)
variance : empirique 3.8865   théorique 4.0
```



Commentaire. Moyenne et variance empiriques sont cohérentes avec $\mu = 5$ et $\sigma^2 = 4$:

- moyenne : l'erreur-type vaut $\sigma/\sqrt{T} = 0{,}02$, et l'écart observé est bien plus petit ;
- variance : pour une gaussienne, la variance empirique a pour écart-type $\sigma^2\sqrt{2/(T-1)} \approx 0{,}057$. L'écart observé (−0,11) vaut 2,01 écarts-types : **juste à la limite** des 95 % ($p \approx 4{,}5\,\%$), comme la moyenne du §1.1. On ne rejette pas au seuil de 1 %, et la moyenne, l'histogramme et le test KS sont bons : c'est une fluctuation (changer la graine donne d'autres valeurs autour de 4). L'histogramme normalisé épouse la cloche théorique centrée en 5, dont la largeur caractéristique vaut $\sigma = 2$.

CE QUE J'AI COMPRIS — Démonstration détaillée ↗

Idée. On vérifie moyenne, variance et forme. Moyenne 4,997 (erreur-type 0,02 : très proche) ; variance 3,89, à 2 écarts-types de 4 : à la limite mais pas anormal. L'histogramme normalisé (aire 1) épouse la cloche de $\mathcal{N}(5, 4)$.

Recul. « Ça ressemble » ne suffit pas : on chiffre les écarts en erreurs-types, et le test KS des tests confirme la loi.

Synthèse des résultats

Graine 2026, $T = 10\,000$ tirages. Source : figures/resultats.json et les sorties des notebooks.

Contrôle	Théorique	Empirique	Écart-type	Écart en écarts-types	p	Verdict
§1.1 · Exponentielle $\mathcal{E}(2)$ par inversion						
Moyenne	0,5000	0,5101	0,0050	2,02	4 %	limite 95 %
Variance	0,2500	0,2604	0,0071	1,47	14 %	normal
Pente semi-log	−2,0000	−1,9586				2,1 % d'erreur
§1.2 · Box-Muller						
Moyenne de $Z = X_1^2 + X_2^2$	2,000	2,019	0,020	0,95	34 %	normal
Corrélation (r^2, θ)	0,0000	−0,0130	0,0100	−1,30	19 %	normal
Corrélation (X_1, X_2)	0,0000	−0,0012	0,0100	−0,12	90 %	normal
Moyenne $\mathcal{N}(5, 4)$	5,0000	4,9973	0,0200	−0,14	89 %	normal
Variance $\mathcal{N}(5, 4)$	4,0000	3,8865	0,0566	−2,01	4 %	limite 95 %

Lecture. « Écart-type » = écart-type théorique de l'estimateur ($\sigma/\sqrt{T}$ pour une moyenne, $1/\sqrt{T}$ pour une corrélation nulle). Jusqu'à 1,96 écart-type, c'est l'intervalle à 95 % ; au-delà de 2,58, on rejetterait au seuil de 1 %. « Limite 95 % » = entre 1,96 et 2,58 : pas de rejet à 1 %. p = probabilité bilatérale d'un écart au moins aussi grand par hasard. Sur une dizaine de contrôles, en voir un ou deux à la limite des 95 % est attendu (environ 1 sur 20 par hasard) ; aucun n'est suspect, et les tests KS de tests/test_generateurs.py confirment les lois.